



EE6405 - Natural Language Processing

NTU Master of Science (MSc) course

Topic A-task 1 FROM TABLE 2

Different Types of NER

NER mainly includes flat, nested or overlapped, and discontinuous NER. In flat NER, entities are continuous and do not overlap, such as "Barack Obama" and "Paris" in a sentence. In nested NER, one entity may appear inside another, such as "Nanyang" within "Nanyang technological university." Discontinuous NER is more complex because the words of one entity may be separated by other words. Flat NER can usually be handled using BIO tagging, while nested and discontinuous NER require more advanced methods.

★

5.0 / 5 ★ (5)

🗨 1

niu come Aug 21, 2026 08:34pm

Different Types of Named Entity Recognition (NER)

(1) Flat NER

Entities are continuous and do not overlap.

Example: Barack Obama visited Paris in 2019.

Entity spans: [Barack Obama] PER [Paris] LOC

BIO tagging: Barack Obama visited Paris in 2019 .
B-PER I-PER O B-LOC O O O

Characteristics

- Continuous spans
- No overlap
- Can be well handled by BIO tagging

(2) Nested / Overlapped NER

One entity may appear inside another, or entities may overlap.

Example: Nanyang technological university is in Singapore

Entity spans: [Nanyang technological university] ORG [Singapore] LOC
[Nanyang] ORG

Characteristics

- Spans may overlap
- Nested is a special case of overlap
- Cannot be properly modeled by BIO tagging

(3) Discontinuous NER

Words of one entity may be separated by other words.

Example: I am having aching in legs and shoulders

Entity spans: [aching in ... shoulders] Symptom

Characteristics

- Non-continuous spans
- Words separated by other tokens
- Requires more advanced methods

PER (Person) ORG (Organization) LOC (Location) Symptom (Entity Type) B-xxx Begin I-xxx Inside O Outside

Topic B - table 16

Task 1

Step 1: Tag the words with labels (Noun, Verbs, etc).

Step 2: Define a chunking pattern that defines how a keyphrase should look like. E.g. Grab any adjective directly followed by a noun/ Grab any verb directly followed by an adverb. Use regex to extract such patterns.

Step 3: Extract the matches from the text.

Task 2

import pandas as pd

import spacy

from nltk import RegexpParser

text = [

"The new wireless headphones provide excellent sound quality.",

"This compact camera takes beautiful photos.",

"The battery lasts surprisingly long.",

"The comfortable keyboard works very well."

]

df = pd.DataFrame(text, columns=["Sentence"])

nlp = spacy.load("en_core_web_sm")

grammar = r"""

NP: {<DT>?<JJ.*>*<NN.*>+}

```

VP: {<MD>?<RB.*>*<VB.*>+<RB.*>*}
"""
chunker = RegexpParser(grammar)
for doc in nlp.pipe(df["Sentence"]):
    print("\nSentence:", doc.text)
    tagged_words = [
        (token.text, token.tag_)
        for token in doc
        if not token.is_punct
    ]
    tree = chunker.parse(tagged_words)
    noun_phrases = []
    verb_phrases = []
    for subtree in tree.subtrees():
        phrase = " ".join(word for word, tag in subtree.leaves())
        if subtree.label() == "NP":
            noun_phrases.append(phrase)
        elif subtree.label() == "VP":
            verb_phrases.append(phrase)
    print("POS tags:", tagged_words)
    print("Noun phrases:", noun_phrases)
    print("Verb phrases:", verb_phrases)
    print(tree)

```

星
3.4 / 5 ★ (5)

🗨 1

star Aug 21, 2026 08:41pm

```

Sentence: The battery lasts surprisingly long.
POS tags: [('The', 'DT'), ('battery', 'NN'), ('lasts', 'VBZ'), ('surprisingly', 'RB'), ('long', 'RB')]
Noun phrases: ['The battery']
Verb phrases: ['lasts surprisingly long']
(S (NP The/DT battery/NN) (VP lasts/VBZ surprisingly/RB long/RB))

Sentence: The comfortable keyboard works very well.
POS tags: [('The', 'DT'), ('comfortable', 'JJ'), ('keyboard', 'NN'), ('works', 'VBZ'), ('very', 'RB'), ('well', 'RB')]
Noun phrases: ['The comfortable keyboard']
Verb phrases: ['works very well']
(S
  (NP The/DT comfortable/JJ keyboard/NN)
  (VP works/VBZ very/RB well/RB))

```

↩ Table 9- Topic A

NER:

Flat NER is unique as in it can't be shared as its own identifier. Where nested NER is linked to another identity and this give it the overlapping of the identifier. Overlapped NER can share more than one word and it doesn't mean it is mutually exclusive.

星
5.0 / 5 ★ (2)

🗨 0

Topic B: Table 24

Task 1:

How is it possible to do keyphrase extraction using POS tagging?

Using grammatical patterns, such as sequences of nouns and adjectives, to identify important phrases in a text.

The pipeline:

1. Tokenize the text into words
2. POS-tag each token (noun, verb, adjective, determiner, etc.)
3. Define a chunk grammar – a regex pattern over tags.
4. Parse with RegexpParser to build a tree grouping matching tag sequences into labeled chunks (NP, VP)
5. Extract the words inside each chunk as your candidate keyphrase

Example:

- Tags: The/DT quick/JJ fox/NN jumps/VBZ
- Grammar <DT>?<JJ>*<NN.*>+ matches The/DT quick/JJ fox/NN
- Extracted keyphrase: "The quick fox"

Task 2:

星

5.0 / 5 ★ (2)

0

```
import re
from nltk import pos_tag, word_tokenize
from nltk.chunk import RegexpParser

sentence = "The quick brown fox jumps over the lazy dog."
tokens = word_tokenize(sentence)
pos_tags = pos_tag(tokens)

# 1. Keyword Extraction (extracting Nouns and Verbs)
keywords = [word for word, tag in pos_tags if tag.startswith('NN') or tag.startswith('VB')]
print("Extracted Keywords:", keywords)

# 2. Regex Chunking for Noun Phrases (NP) and Verb Phrases (VP)
# NP pattern: Optional determiner, any number of adjectives, followed by a noun
# VP pattern: Verb, followed by optional noun phrase
grammar = """
NP: <DT>?<JJ>*<NN.*>+
VP: <VB.*><NP>?
"""
chunk_parser = RegexpParser(grammar)
chunk_tree = chunk_parser.parse(pos_tags)

print("\nChunked Phrases:")
for subtree in chunk_tree.subtrees():
    if subtree.label() == 'NP' or subtree.label() == 'VP':
        print(f"{subtree.label()}: {' '.join(word for word, pos in subtree.leaves())}")

✓ 0.7s
Extracted Keywords: ['brown', 'fox', 'jumps', 'dog']

Chunked Phrases:
NP: The quick brown
NP: fox
VP: jumps
NP: the lazy dog
```

TaskA: table 12

task1: Flat NER refers to the simplest form of named entity recognition, where each entity is represented by a continuous span of text and different entities cannot overlap with each other. In this setting, each token usually belongs to at most one entity. For example, in “Barack Obama was born in Hawaii,” “Barack Obama” is a PERSON entity and “Hawaii” is a LOCATION entity. Flat NER can be conveniently modeled using BIO or BIOES tagging and is widely used in general-domain NER tasks. Nested NER allows one entity to be completely contained inside another entity. For example, in “University of Cambridge,” “Cambridge” can be recognized as a LOCATION, while “University of Cambridge” can simultaneously be recognized as an ORGANIZATION. This means that the same token may belong to more than one entity. Standard BIO tagging is usually not sufficient for nested NER, so span-based or layered approaches are often used. Overlapping NER allows two entity mentions to share some tokens. Unlike nested NER, the overlap does not necessarily involve one entity being fully contained within another. One entity may cover tokens A-B-C while another covers B-C-D. This structure is more complex than flat NER and usually requires span-based, graph-based, or other structured prediction methods. Discontinuous NER allows a single entity mention to consist of multiple non-contiguous pieces of text. In other words, parts of the same entity can be separated by unrelated words. This type of NER is especially common in biomedical and clinical texts, where an entity may be expressed across several separated spans. Because the entity is not a single continuous sequence, standard sequence-labeling methods such as BIO tagging cannot represent it easily, and more complex graph-based or structured methods are often required. Overall, flat NER is the simplest structure, while nested, overlapping, and discontinuous NER allow increasingly complex relationships between entity spans. The main differences lie in whether entities can overlap, contain one another, or consist of separated text segments.

task2: W2NER is a unified NER model proposed in an AAAI 2022 paper. It handles flat, nested/overlapped, and discontinuous NER by converting entity recognition into word-word relation classification. The GitHub code implements this model using a 2D word-pair grid, relation labels such as NNW and THW-*, and neural components for relation prediction.

星

5.0 / 5 ★ (3)

🗲 0

↩ Table 21 Topic B

For example:

The powerful language model generates high-quality text.

POS tagging gives approximately:

The/DT
powerful/JJ
language/NN
model/NN
generates/VBZ
high-quality/JJ
text/NN

星

0 / 5 ★

🗲 0

↩ Table40 (Topic C)

task1:B(took the lesson)"to heart" in VP, "the lesson" is NP

task2:e&f

“garbage and junk” inside “of”, “into a bin” attaches up as VP.

e and f just drawn flat vs nested

星

5.0 / 5 ★ (4)

🗲 0

↩ Topic B-Table 21

```
import nltk
from nltk import word_tokenize, pos_tag
from nltk.chunk import RegexpParser

sentence = "The intelligent system extracts useful
information efficiently."

tokens = word_tokenize(sentence)
tagged = pos_tag(tokens)

grammar = r"""
NP: {<DT>?<JJ>*<NN.*>+}
VP: {<VB.*><RB.*>*}
"""

chunk_parser = RegexpParser(grammar)
result = chunk_parser.parse(tagged)

print(result)

for subtree in result.subtrees():
    if subtree.label() in ["NP", "VP"]:
        phrase = " ".join(word for word, tag in
```

```
NP : The intelligent system
VP : extracts
NP : useful information
```

Task 2

```
subtree.leaves()
print(subtree.label(), ":", phrase)
```

星

0 / 5 ★

0

⇒ Topic C From Table 31

Task 1 Analysis & Solution

Sentence: "She took the lesson to heart"

Grammar Breakdown:

- **Subject (NP):** She (PRP)
- **Verb (VBD):** took
- **Direct Object (NP):** the lesson (DT + NN)
- **Prepositional Phrase / Complement (PP):** to heart (IN + NN)
- **Structure:** In standard constituency parsing, the verb took directly governs both the direct object the lesson and the idiom complement to heart. Therefore, the VP splits into three branches: **VP -> VBD NP PP**.

Option Comparison:

- **A:** Attaches to heart under lesson (NP -> NP PP), which incorrectly modifies "lesson" rather than the verb "took".
- **B (Correct):** Correctly branches VP into VBD (took), NP (the lesson), and PP (to heart) at the same level.
- **C:** Incorrect nested structure.
- **D:** Incorrectly flattens the root node S into four parallel branches, removing the VP level.

Correct Answer: B

Task 2 Analysis & Solution

Sentence: "workers dumped sacks of garbage and junk into a bin"

Key Structural Point: Notice that in trees **(d)** and **(e)**, the part-of-speech (POS) tags and syntactic structures follow specific conventions:

- **Coordination:** They group garbage and junk together under a **CNP** (Conjoined Noun Phrase) or handle and properly as a coordinator (C).
- **Prepositional Attachment:**
 - Tree (d):** Attaches the PP (into a bin) at the **VP level** (modifying the action *dumped*).
 - Tree (e):** Attaches the PP (into a bin) under the **NP/PP level** (modifying what is in the bin).

Since Task 2 explores syntactic ambiguity in phrase structure parsing (how PP attachment and conjunctions interact), both structural variations **(d)** and **(e)** represent valid parse trees under this grammar convention.

Correct Answers: (d), (e)

The code is here:

```
"""EE6405 Topic C classroom discussion: constituency-tree verification.

The slide calls the exercise dependency parsing, but its candidate structures
are phrase-structure trees (S/NP/VP/PP/CNP). This script deliberately uses a
small standard-library bracket parser instead of treating a dependency parser
as the gold answer. It checks the terminal order and prints the two
English-compatible parses selected in the accompanying analysis.
"""

from __future__ import annotations

from dataclasses import dataclass
import re
import sys
from typing import Iterable

TOKEN_RE = re.compile(r"\\(|\\)|\\[\\^s()]+")

@dataclass
class Node:
    label: str
    children: list["Node"]

    def leaves(self) -> list[str]:
```

```

    if not self.children:
        return [self.label]
    result: list[str] = []
    for child in self.children:
        result.extend(child.leaves())
    return result

def pretty(self, level: int = 0) -> str:
    indent = " " * level
    if not self.children:
        return f"{indent}{self.label}"
    lines = [f"{indent}{self.label}"]
    lines.extend(child.pretty(level + 1) for child in self.children)
    return "\n".join(lines)

def parse_tree(text: str) -> Node:
    tokens = TOKEN_RE.findall(text)
    position = 0

    def parse_node() -> Node:
        nonlocal position
        if position >= len(tokens) or tokens[position] != "(":
            raise ValueError(f"Expected '(' at token {position}: {tokens[position:position + 3]}")
        position += 1
        if position >= len(tokens):
            raise ValueError("Missing node label")
        label = tokens[position]
        position += 1
        children: list[Node] = []
        while position < len(tokens) and tokens[position] != ")":
            if tokens[position] == "(":
                children.append(parse_node())
            else:
                children.append(Node(tokens[position], []))
                position += 1
        if position >= len(tokens):
            raise ValueError(f"Missing ')' for node {label}")
        position += 1
        return Node(label, children)

    tree = parse_node()
    if position != len(tokens):
        raise ValueError(f"Unexpected trailing tokens: {tokens[position:]}")
    return tree

```

```

SENTENCE = "workers dumped sacks of garbage and junk into a bin".split()

```

```

# These are the two parses that are compatible with ordinary English lexical
# selection: dump takes the dumped NP and a destination PP; the PP of garbage
# is inside the NP. Their only difference is the scope of and.

```

```

TREES = {
    "(d)": """
(S
  (NP workers)
  (VP
    (V dumped)
    (NP
      (NP (NP sacks) (PP (P of) (NP garbage)))
      (C and)
      (NP junk))
    (PP (P into) (NP (D a) (N bin))))
""",
    "(e)": """
(S
  (NP workers)
  (VP
    (VP
      (V dumped)
      (NP
        (NP sacks)
        (PP

```

```

        (P of)
        (NP (NP garbage) (CNP (C and) (NP junk))))))
    (PP (P into) (NP (D a) (N bin))))
    """,
}

def verify(name: str, text: str) -> None:
    tree = parse_tree(text)
    leaves = tree.leaves()
    print(f"{name}: root={tree.label}; leaves_ok={leaves == SENTENCE}")
    print(f"  leaves: {' '.join(leaves)}")
    if leaves != SENTENCE:
        raise AssertionError(f"{name} does not yield the target sentence")
    print("  tree:")
    print(tree.pretty())

if __name__ == "__main__":
    print(f"Python executable: {sys.executable}")
    print(f"Target sentence: {' '.join(SENTENCE)}")
    for tree_name, tree_text in TREES.items():
        verify(tree_name, tree_text)

```

星
2.0 / 5 ★ (1)

0

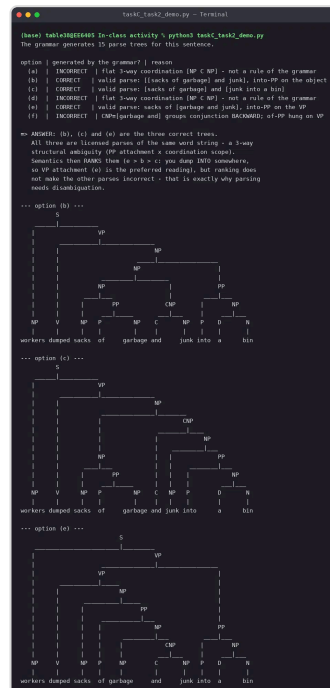
Table 38 TaskC

Task 1 — Answer: B

The key question is where the prepositional phrase "to heart" attaches. "Take something to heart" is a verb idiom: *to heart* describes the action of taking, so the PP must attach at the **VP level**, as a sister of the object NP: VP → VBD took + NP [*the lesson*] + PP [*to heart*]. That is exactly tree **B**. Why the others are wrong: in **A** and **C** the PP attaches to the noun *lesson*, wrongly claiming "*the lesson to heart*" is a constituent (a lesson "belonging to heart" — no such meaning). In **D** the PP attaches directly under S, outside the VP, wrongly treating *to heart* as a sentence-level modifier instead of part of the predicate.

Task 2 — Answer: (b), (c) and (e)

As the question suggests, we used Python to verify our answer: we wrote the CFG that the six trees themselves imply (labels S, NP, VP, PP, CNP, with coordination handled by the binary rules NP → NP CNP and CNP → C NP), and let NLTK's ChartParser generate all parse trees of the sentence — 15 in total. We then encoded the six candidate trees exactly as drawn and checked each one against the parser's output: exactly three of them, (b), (c) and (e), are generated by the grammar, and they are the three correct answers, corresponding to a genuine three-way structural ambiguity in PP attachment and coordination scope — in (b) the coordination is [[sacks of garbage] and junk] with the PP "into a bin" attached to the object NP, in (c) the two conjuncts are [sacks of garbage] and [junk into a bin], and in (e) the sacks contain [garbage and junk] and "into a bin" attaches to the VP. The other three fail on pure syntax: (a) and (d) use a flat three-way coordination [NP C NP] in a single constituent, which is not a rule of this grammar (a



conjunction must first combine with the following conjunct to form a CNP), and (f) groups the conjunction backwards into [garbage and] and hangs the "of"-PP off the verb, although an "of"-phrase can only modify a noun. Note that all three correct trees describe the same word string but different meanings; semantics only ranks them — (e) is the preferred reading, since you dump something into somewhere — but ranking does not make the other licensed parses incorrect, which is precisely why parsers need disambiguation.

星

5.0 / 5 ★ (3)

🗨 1

Zhifei Song Aug 21, 2026 08:34pm
wonderful

⇒ Topic B - table 28

Task 1: Keyphrase extraction with POS tagging

POS tagging assigns a grammatical label to each word, such as NN for noun, JJ for adjective, VB for verb, and DT for determiner. Regular-expression chunking can then combine POS-tagged words into phrases.

Common rules:

NP: {<DT>?<JJ.><NN.*>+}

This extracts noun phrases, such as “the wireless mouse” and “excellent battery life”.

VP: {<RB.><VB.>+<RB.>*}

This extracts verb phrases, such as “easily installed” and “works well”.

PP: {<IN><NP>}

This extracts prepositional phrases, such as “in low light” and “on my desk”.

Noun phrases are useful keyphrase candidates because they usually represent important objects, features, or topics in a text.

Task 2: Demonstration code

```
import nltk
```

```
from nltk import word_tokenize, pos_tag, RegexpParser
```

```
nltk.download("punkt")
```

```
nltk.download("averaged_perceptron_tagger")
```

```
reviews = [
```

```
    "The excellent battery life is worth the high price.",
```

```
    "I easily installed the wireless mouse on my laptop.",
```

```
    "Customer service responded very quickly.",
```

```
    "The camera takes sharp pictures in low light."
```

```
]
```

```
grammar = r"""
```

```
NP: {<DT>?<JJ.><NN.>+}
```

```
VP: {<RB.><VB.>+<RB.>*}
```

```
PP: {<IN><NP>}
```

```
"""
```

```
parser = RegexpParser(grammar)
```

```
for review in reviews:
```

```
    tokens = word_tokenize(review)
```

```
    tagged_words = pos_tag(tokens)
```

```
    tree = parser.parse(tagged_words)
```

```
    print("\nReview:", review)
```

```
    print("POS tags:", tagged_words)
```

```
    for phrase_type in ["NP", "VP", "PP"]:
```

```
        phrases = []
```

```
        for subtree in tree.subtrees(
```

```
            filter=lambda t: t.label() == phrase_type
```

```
        ):
```

```
            phrase = " ".join(word for word, tag in subtree.leaves())
```

```
            phrases.append(phrase)
```

```
    print(phrase_type + ":", phrases)
```


Table 42

Topic C:

Task 1:

The Answer is B. Because 'She' is the subject Noun Phrase. The Verb Phrase contains the verb 'took', object Noun Phrase 'the lesson', and Prepositional Phrase 'to heart'.

Task 2:

The answer is d and e. Both keep 'into a bin' attached to the VP headed by 'dumped'. They differ in coordination scope ([sack of garbage] and [junk])/(sacks of [garbage and junk]). The other candidates attach into 'a bin' inside the object Noun Phrase, attach it only to 'junk', or incorrectly attach 'of garbage and junk' to the Verb Phrase instead of to 'sacks'.

Table 26 - Topic B

Chunking means grouping words together into meaningful phrases based on their POS tags.

Wind Surfer Aug 21, 2026 08:32pm
excellent!

```
1 import nltk
2 from nltk import word_tokenize, pos_tag, RegexpParser
3 from collections import Counter
4
5 #
6 # 1. Download required NLTK resources
7 #
8 nltk.download("punkt")
9 nltk.download("punkt_tab")
10 nltk.download("averaged_perceptron_tagger")
11 nltk.download("averaged_perceptron_tagger_eng")
12
13 #
14 # 2. Sample dataset
15 #
16 reviews = [
17     "The new phone has an excellent camera and long battery life.",
18     "The screen looks very bright and works extremely well.",
19     "Customers love the fast processor and smooth performance.",
20     "The lightweight laptop provides excellent performance.",
21     "The speakers are surprisingly good and produce clear sound."
22 ]
23
24 #
25 # 3. Define regular-expression chunking rules
26 #
27 # NP:
28 # optional determiner + zero or more adjectives + one or more nouns
29 np_grammar = r"""
30 NP: (<DT|PRP|$>+<JJ>+<NN>+<N>+)
31
32 # VP:
33 # optional modal + verb + zero or more adverbs/verbs/adjectives
34 vp_grammar = r"""
35 VP: (<VB>+<VB>+<RB>+<VB>+<JJ>+<N>+)
36
37 # Adjective Phrase:
38 # zero or more adverbs + one or more adjectives
39 adjp_grammar = r"""
40 ADJP: (<RB>+<JJ>+<N>+)
41
42 #
43 # 4. Function to extract one type of phrase
44 #
45 def extract_chunk(tagged_sentence, grammar, label):
46     parser = RegexpParser(grammar)
```

Padlet Drive

week2_code

Topic A-task 1 FROM TABLE 6

1. Flat: Entities don't overlap or nest. Each word belongs to at most one entity. Example: "John works at NTU." → "John" (PERSON) and "NTU" (ORGANIZATION) are two separate, non-overlapping entities. 2. Nested: One entity is fully contained inside another entity. Example: "[University of Washington]" → "Washington" (LOCATION) is nested inside "University of Washington" (ORGANIZATION). 3. Discontinuous: A single entity is made up of non-adjacent word spans. Example: "pain in left and right knee" → "left knee" and "right knee" are each one entity, but their words are not next to each other in the sentence.

Topic C-table 30

task1

B

The phrase "to heart" belongs to the verb "took" (this is the fixed idiom "take something to heart"), so the PP "to heart" needs to attach under the VP (verb phrase).

task2

D works because it breaks the sentence down the right way: workers + dumped + sacks of garbage and junk + into a bin. The "into a bin" part sits with the verb (it describes where they dumped it), not with the trash. And "sacks of garbage" and "junk" are properly joined by "and."

E works too— it's just a different shape. It uses a two-layer VP: the inner layer is dumped sacks of garbage and junk, and the outer layer adds into a bin. The "garbage and junk" part is correctly grouped together inside the "of" phrase. It's another valid way to draw the same sentence.

星

5.0 / 5 ★ (1)

0

TopicB-Table21

```
import nltk
from nltk.corpus import stopwords
from collections import Counter

nltk.download('punkt')
nltk.download('punkt_tab')
nltk.download('averaged_perceptron_tagger')
nltk.download('averaged_perceptron_tagger_eng')
nltk.download('stopwords')

class KeyphraseExtractor:
    def __init__(self):
        self.stop_words = set(stopwords.words('english'))
        self.grammar = r"""
            NP: {<DT>?<JJ>*<NN>+}
            VP: {<VB.*><NP|PP>+}
            """
        self.chunk_parser = nltk.RegexpParser(self.grammar)

    def extract(self, text, phrase_type='NP'):
        tokens = nltk.word_tokenize(text)
        pos_tags = nltk.pos_tag(tokens)
        chunked = self.chunk_parser.parse(pos_tags)

        phrases = []
        for subtree in chunked.subtrees(filter=lambda t: t.label() == phrase_type):
            words = [word for word, tag in subtree.leaves()]
            if len(words) >= 2 and not all(w.lower() in self.stop_words for w in words):
                phrases.append(" ".join(words))

        seen = set()
        unique = []
        for p in phrases:
            if p.lower() not in seen:
                seen.add(p.lower())
                unique.append(p)
        return unique

extractor = KeyphraseExtractor()

texts = [
    "The wireless headphone has amazing sound quality.",
    "The battery life is incredibly long.",
```

提取名词短语 (NP):

Text 1: The wireless headphone has amazing sound quality.
Phrases: ['The wireless headphone', 'amazing sound quality']

Text 2: The battery life is incredibly long.
Phrases: ['The battery life']

Text 3: The noise cancellation feature blocks background noise.
Phrases: ['The noise cancellation feature']

频次统计:

The wireless headphone: 1次
amazing sound quality: 1次
The battery life: 1次
The noise cancellation feature: 1次

```

"The noise cancellation feature blocks
background noise."
]

print("=" * 50)
print("提取名词短语 (NP):")
print("=" * 50)

for i, text in enumerate(texts, 1):
    phrases = extractor.extract(text, 'NP')
    print(f"Text {i}: {text}")
    print(f"Phrases: {phrases}\n")

all_phrases = []
for text in texts:
    all_phrases.extend(extractor.extract(text,
'NP'))

print("=" * 50)
print("频次统计:")
print("=" * 50)
counter = Counter(all_phrases)
for phrase, count in counter.most_common(5):
    print(f"    {phrase}: {count}次")

```

星

0 / 5 ★

🗨 0

↩ Table 35 Topic C

Task 1: B

The tree starts with ‘S’ (sentence) and is broken down into a ‘NP’ (noun phrase = ‘She’) and ‘VP’ (verb phrase = the remaining sentence, ‘took the lesson to heart’), which is then broken up further into a verb VBD ‘took’, and another NP and PP

星

0 / 5 ★

🗨 0

↩ Table 5 -Topic A

Task 1:

There are 3 major types of NE: flat, overlapped, and discontinuous NER

1. Flat NER means the words refer to only one entity and entity words are continuous with no overlaps (eg, "New York" is one location)
2. Overlapped NER means entities sit inside other entities (eg "Bank of America" is an organization, but "America" inside it is also a location on its own).
3. Discontinuous NER means an entity's words can have gaps between them. (eg "The pain I was experiencing around the hip joints was incredible." but the target entity: "pain hip joints" (a medical symptom)

Task B:

The paper combines all three types of NER (flat, nested, discontinuous) into one model.

Instead of giving each word a label, it looks at every pair of words and asks: are these two words connected in an entity? It uses two simple relations — NNW ("this word comes next in the entity") and THW ("this word is the end, that word is the start, and the entity type is X"). For example, in "aching in legs and shoulders," there are two symptom entities sharing the words "aching in." Normal tagging fails here because "in" can only get one label. W²NER handles it easily — "in" just points to both "legs" and "shoulders" as next words, so both entities are captured at once. This one model beat all specialised models across 14 datasets covering all three NER types.

星
0 / 5 ★

0

⇒ table 22 task B

Task 1 – Why POS tagging works

Keyphrases are often noun phrases such as battery life and camera quality. POS tags let us keep those patterns and ignore less other words with less information.

Noun phrase <JJ>*<NN.*>+ battery life

Verb phrase <VB.*><RB>*<JJ>* works perfectly

Task 2

"The excellent battery life and fast charging make this phone stand out."

```
import nltk
```

```
from nltk import word_tokenize, pos_tag, RegexpParser
```

```
text = "The excellent battery life and fast charging make this phone stand out."
```

```
# Download once if needed
```

```
# nltk.download("punkt")
```

```
# nltk.download("averaged_perceptron_tagger")
```

```
tokens = word_tokenize(text)
```

```
tagged = pos_tag(tokens)
```

```
grammar = r"""
```

```
NP: {<JJ.*>*<NN.*>+}
```

```
VP: {<VB.*><RB.*>*<JJ.*>*}
```

```
"""
```

```
tree = RegexpParser(grammar).parse(tagged)
```

```
for subtree in tree:
```

```
    if hasattr(subtree, "label"):
```

```
        phrase = " ".join(word for word, _ in subtree.leaves())
```

```
        print(subtree.label(), ":", phrase)
```

```
output
```

```
NP : excellent battery life
```

```
NP : fast charging
```

```
VP : make
```

```
NP : phone
```

星
5.0 / 5 ★ (1)

1

Fiona Aug 21, 2026 08:37pm
great

⇒ Tabel27-Taskb

T1: use POS to assign tags → define the normal combination of

VP/NP → turn into regular expressions → extract the keyphrase then 🍌🍌🍌

T2: import nltk

```
from nltk import word_tokenize, pos_tag
```

```
from nltk.chunk import RegexpParser
```

```
sentence = "The brave student solved the difficult problem."
```

```
# POS tagging
```

```
words = word_tokenize(sentence)
```

```
tags = pos_tag(words)
```

```
print(tags)
```

```
# Chunking rules
```

```
grammar = r"""
```

```
NP: {<DT>?<JJ>*<NN.*>+}
```

```
VP: {<VB.*><RB>{*}
```

```
"""
```

```
parser = RegexpParser(grammar)
```

```
tree = parser.parse(tags)
print(tree)
```

星

5.0 / 5 ★ (2)

🗨 0

↩ Table 4 -Topic A

Task 1:

NER can be **flat**, where entity spans are continuous and cannot overlap; **nested**, where one entity is contained completely inside another; **overlapping**, where spans intersect (sharing tokens); or **discontinuous**, where one entity consists of multiple separated text spans. It can also use coarse-, fine-, or domain-specific entity labels, with more specialized systems recognizing hierarchical or previously unseen entity types. i.e. “New York University”, for flat, it identifies as a whole as **Organisation**; for nested, “New York” is identified as **Location** inside “New York University” (**Organisation**); for overlapping, “New York University Hospital” contains “New York”(**Location**) and “University Hospital”(**Facility**).

星

0 / 5 ★

🗨 2

Zenith Reacher Aug 21, 2026 08:36pm

The article introduces W2NER, a unified named entity recognition (NER) framework designed to handle flat, nested, and discontinuous entities within a single model. Unlike span-based approaches that primarily focus on identifying entity boundaries, or sequence-to-sequence methods that can suffer from exposure bias, W2NER reformulates NER as a word-to-word relation classification task. It represents sentences as a two-dimensional grid of word pairs and uses Next-Neighboring-Word (NNW) and Tail-Head-Word (THW) relations to capture connections between entity words. Multi-granularity 2D convolutions are applied to refine the grid representations, while a co-predictor is used to reason about the relationships between words. Experiments across 14 benchmark datasets covering English and Chinese NER demonstrate that W2NER outperforms existing state-of-the-art methods for flat, nested, and discontinuous NER.

Ruizhi He Aug 21, 2026 08:40pm

In simple words, instead of asking "What label should EACH WORD get?", it asks "What relationship exists between EACH PAIR OF WORDS?"

↩ Topic C-table 32

Task 1: The correct answer is **B**. The sentence “*She took the lesson to heart*” has the structure **S → NP + VP**: **She** is an NP, **took** is a V, **the lesson** is an NP, and **to heart** is a PP. Therefore, the VP has the structure **V + NP + PP**. A and C incorrectly place *to heart* inside the NP *the lesson*, while D incorrectly attaches the PP directly to the S level.

Task 2: The correct answers are **A, B, and D**, as they represent different valid syntactic analyses of the sentence. In A, *sacks of [garbage and junk]* means that **garbage and junk are coordinated complements of “of”**. In B and D, the structure is **[sacks of garbage] and [junk]**, meaning that two NPs are coordinated. C incorrectly attaches *into a bin* to *junk*, E incorrectly attaches *into a bin* directly to S, and F incorrectly treats *of garbage and junk* as a VP-level modifier instead of part of the NP *sacks*.

星

0 / 5 ★

🗨 0

Topic C: group 31

Task1: B

For Task 1, “She took the lesson to heart,” the correct structure is:
She + [took + the lesson + to heart]
“The lesson” is the object of took, while
“to heart” is a prepositional phrase attached to the VP. Therefore, B is correct.

Task2: D E

For Task 2, “workers dumped sacks of garbage and junk into a bin,” there are two possible interpretations:

D: workers dumped [sacks of garbage] and [junk] into a bin.

E: workers dumped sacks of [garbage and junk] into a bin.

Both parses are grammatically possible, so the correct answers are D and E.



星

0 / 5 ★

0 0

Table 36 - C

Task 1: b

task 2: d and e

星

0 / 5 ★

0 0

Topic A Table 3

Task 1:

1. Flat NER identifies entities that do not overlap with each other; for example, in “John works at Google,” John is a Person and Google is an Organization.
 2. Nested NER allows one entity to appear inside another entity; for example, in “University of Cambridge,” Cambridge can be a Location while University of Cambridge is an Organization.
 3. Overlapping NER allows two entities to share some of the same words without one being fully inside the other; for example, “New York University” and “University Hospital” share the word “University.”
 4. Discontinuous NER identifies one entity that is made up of words that are separated in the sentence; for example, “pain in the lower back” may be treated as one medical entity.
 5. Unified NER uses one model or framework to handle different NER structures, such as flat, nested, overlapping, and discontinuous entities, instead of using a separate model for each type; for example, the same system can recognize both “Google” as a flat entity and “University of Cambridge” as a nested entity.
- Note: Nested NER is a special type of overlapping NER. All nested entities overlap, but not all overlapping entities are nested.

星

5.0 / 5 ★ (1)

0 0

Table 20 TaskB

POS tagging labels each token according to its grammatical role, for example ‘excellent/JJ battery/NN life/NN’. This preserves structure that bag-of-words methods discard. Chunking (shallow parsing) then groups consecutive tagged tokens into phrases with an interpretable regular expression and returns a tree.

The workflow implemented here is:

1. Normalise the review: decode HTML entities, remove HTML tags, and collapse

- whitespace. The article lists lower-casing as a normalisation option, but this implementation deliberately retains case until after POS tagging because it improves an English tagger's treatment of sentence-initial pronouns and proper nouns.
2. Tokenise it with ``nltk.word_tokenize()``.
 3. POS-tag tokens with ``nltk.pos_tag()``.
 4. Run ``nltk.RegexpParser()`` separately for each POS grammar.
 5. Read matching tree subtrees back as original word phrases, then lower-case, deduplicate, and rank candidates by frequency.

星
0 / 5 ★

0

group 1 - topic A

Task 1:

Flat NER - entities cannot overlap or contain other entities

Nested NER - one entity can be inside another entity

Overlapping NER - entity spans can share words, without necessarily being fully contained

Discontinuous NER - one entity consists of multiple separated pieces of text

Multi-label NER - the same span can have multiple entity types

Task 2:

real text contain more complicated entities than flat entities, having separate approaches for each is inconvenient. the authors created one model, W2NER that can handle all 3 types simultaneously. they do not treat NER as BIO sequence tagging but rather a word-to-word relation classification problem. the authors represent a sentence using a two-dimensional word-pair grid. the model predicts next-neighbouring-word (NNW) relations to indicate which words belong consecutively to the same entity and tail-head-word (THW) relations to identify the beginning, end and type of entity.

星
0 / 5 ★

0

Table 18

We think the basic idea of using POS tagging for keyphrase extraction is quite straightforward. First, we assign a part-of-speech tag to each word, such as noun, adjective, verb, or preposition. After that, we define some common phrase patterns based on these POS tags and use regular expressions to perform chunking.

```
import nltk
from nltk.tokenize import word_tokenize
from nltk.tag import pos_tag
from nltk.chunk import RegexpParser
# Download required NLTK resources (run once)
nltk.download('punkt')
nltk.download('averaged_perceptron_tagger')
# Sample dataset (simulating Amazon-style reviews)
reviews = [
    "The battery life of this phone is amazing.",
    "Camera quality is superb and the display is very bright.",
    "It has a smooth touch response and fast performance."
]
# Define POS-based chunking grammar
grammar = r"""
    NP: {<DT>?<JJ>*<NN>.*>}      # Noun Phrase
    VP: {<VB>.*<RB>*<JJ|NN>.*>}    # Verb Phrase
    ADJ_NN: {<JJ><NN>.*>}          # Keyphrase: Adjective + Noun
    """
parser = RegexpParser(grammar)
# Process each review
for i, text in enumerate(reviews, 1):
```

```
print(f"\nReview {i}: {text}")
print("-" * 60)
# Tokenization & POS tagging
tokens = word_tokenize(text)
pos_tags = pos_tag(tokens)
# Chunking
tree = parser.parse(pos_tags)
# Extract and display phrases
for subtree in tree.subtrees():
    if subtree.label() != "S":
        phrase = " ".join([word for word, tag in subtree.leaves()])
        print(f"{subtree.label():<7s}: {phrase}")
```

星
0 / 5 ★
0

Table10

Task1
Flat NER handles non-overlapping entities. Nested NER allows one entity inside another, while discontinuous NER allows non-adjacent spans to form one entity.
Task2

W2NER converts NER into word-word relation classification. Its two-dimensional word-pair grid can represent flat, nested and discontinuous entities in one unified framework.

星
0 / 5 ★
0

Table3 TopicA Task2

星
5.0 / 5 ★ (1)
0

EE6405 W2 TABLE3 TOPIC1 TASK2

EE6405 W2 TABLE3 TOPIC1 TASK2

W2NER treats NER as **word-pair relation classification** instead of token labeling. It builds a 2D grid of word pairs and predicts how words connect, allowing one model to handle **flat, nested, and discontinuous entities**.

Main Relation Model use
NNW (Next-Neighboring-Word): identifies whether two words are connected as neighboring words within an entity.
THW (Tail-Head-Word): connects the end of an entity back to its beginning and also identifies the entity type.

The diagram illustrates the W2NER architecture. It starts with an input sequence of words: "I", "am", "having", "aching", "in", "legs", "and", "shoulders". These words are processed by an **Encoder Layer** (BERT). The output is a **Word Embedding** matrix. This matrix is then processed by a **Convolution Layer** (BERT Style Grid Representation) which uses **Multi-Granularity Dilated Convolution** to produce **Distance-1 Encoding**, **Distance-2 Encoding**, and **Distance-3 Encoding**. These encodings are then processed by a **Co-Predictor Layer** (CLN and MLP) to produce **Word-Word Relation Classification**. The final output is a **Decoding** step that produces the final entity spans: "aching in legs" and "aching in shoulders".

Figure 3: Overall NER architecture. CLN and MLP represent conditional layer normalization and multi-layer perceptron. ⊕ and ⊗ represent element-wise addition and concatenation operations.

W2NER's pipeline: BERT + BiLSTM encode words, a 2D CNN models word-pair relations, and biaffine + MLP predict the relations used to decode entities.

Github Architecture: BERT → BiLSTM → conditional layer normalization + distance/region embeddings → multi-dilation 2D CNN → combined biaffine + MLP predictor.

Application
Biomedical and Clinical Domain
the authors state that the datasets they used to test "discontinuous NER" (CADEC, ShARE) are derived from clinical documents. This shows that a direct application of their model is extracting medical terms, symptoms, and adverse drug reactions from doctors' notes and medical reports.



topic B -- Table 25

Keyphrase extraction using part-of-speech tagging works by identifying words according to their grammatical roles and then combining words that follow useful grammatical patterns. POS tagging assigns labels such as noun, verb, adjective, adverb, and determiner to each token in a sentence. Since important concepts in a text are often expressed as noun phrases or verb phrases, POS tags can be used to locate these structures and extract them as candidate keyphrases. For example, expressions such as “machine learning model,” “powerful processor,” and “customer service team” contain nouns and adjectives that together represent meaningful concepts. Extracting the complete phrase is usually more informative than treating each word separately. One common way to perform this extraction is through chunking with regular expressions. After a sentence has been tokenized and POS-tagged, a regular-expression grammar can define the sequence of POS tags that should be treated as a phrase. For example, the pattern `<DT>?<JJ.*>*<NN.*>+` can be used to identify noun phrases. In this expression, `<DT>?` means that an optional determiner may appear at the beginning, `<JJ.*>*` allows zero or more adjectives, and `<NN.*>+` requires one or more nouns. Therefore, a phrase such as “the powerful new language model” can be detected because it follows the pattern determiner + adjectives + nouns. Similarly, noun-noun combinations such as “customer service” or adjective-noun combinations such as “powerful processor” can also be extracted as candidate keyphrases.

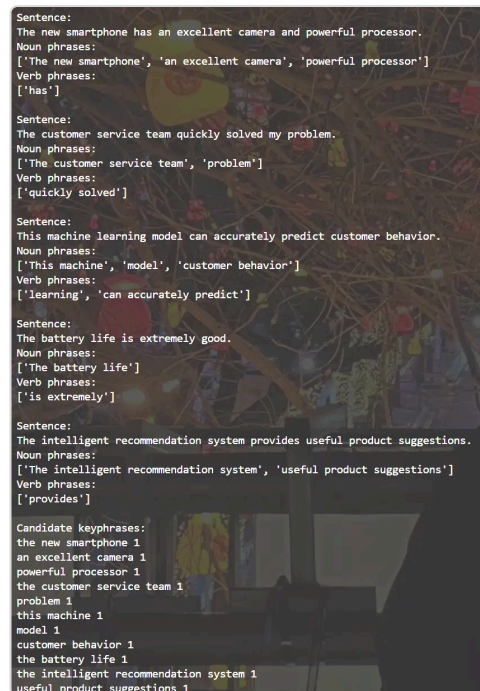
Verb phrases can be extracted in a similar way by defining patterns that include verbs, modal verbs, and adverbs. For example, a pattern such as `<MD>?<RB.*>*<VB.*>*<RB.*>*<VB.*>*` can identify phrases such as “can accurately predict” or “quickly solved.” In this way, POS tagging provides the grammatical information, while regular-expression chunking groups related words into meaningful units. The overall process can therefore be described as text tokenization, POS tagging, phrase chunking, candidate phrase extraction, and finally filtering or ranking the extracted phrases. This method is useful for keyphrase extraction because it preserves meaningful multi-word expressions instead of selecting isolated words only. As a result, phrases such as “machine learning model” can be retained as a complete semantic unit rather than being separated into individual words such as “machine,” “learning,” and “model.”

code:

```
import nltk
from nltk import word_tokenize, pos_tag
from nltk.chunk import RegexpParser
from collections import Counter

# Download required NLTK resources
nltk.download("punkt")
nltk.download("punkt_tab")
nltk.download("averaged_perceptron_tagger")
nltk.download("averaged_perceptron_tagger_eng")

# -----
```



```
Sentence:
The new smartphone has an excellent camera and powerful processor.
Noun phrases:
['The new smartphone', 'an excellent camera', 'powerful processor']
Verb phrases:
['has']

Sentence:
The customer service team quickly solved my problem.
Noun phrases:
['The customer service team', 'problem']
Verb phrases:
['quickly solved']

Sentence:
This machine learning model can accurately predict customer behavior.
Noun phrases:
['This machine', 'model', 'customer behavior']
Verb phrases:
['learning', 'can accurately predict']

Sentence:
The battery life is extremely good.
Noun phrases:
['The battery life']
Verb phrases:
['is extremely']

Sentence:
The intelligent recommendation system provides useful product suggestions.
Noun phrases:
['The intelligent recommendation system', 'useful product suggestions']
Verb phrases:
['provides']

Candidate keyphrases:
the new smartphone 1
an excellent camera 1
powerful processor 1
the customer service team 1
problem 1
this machine 1
model 1
customer behavior 1
the battery life 1
the intelligent recommendation system 1
useful product suggestions 1
```

```

# 1. Sample dataset
# -----

reviews = [
    "The new smartphone has an excellent camera and
    powerful processor.",
    "The customer service team quickly solved my
    problem.",
    "This machine learning model can accurately
    predict customer behavior.",
    "The battery life is extremely good.",
    "The intelligent recommendation system provides
    useful product suggestions."
]

# -----
# 2. Define POS regex patterns
# -----

grammar = r"""
    NP: {<DT>?<JJ.*>*<NN.*>+}
    VP: {<MD>?<RB.*>*<VB.*>*<RB.*>*<VB.*>*}
    """

chunk_parser = RegexpParser(grammar)

# -----
# 3. Extract noun and verb phrases
# -----

def extract_phrases(sentence):

    tokens = word_tokenize(sentence)

    tagged_words = pos_tag(tokens)

    chunk_tree = chunk_parser.parse(tagged_words)

    noun_phrases = []
    verb_phrases = []

    for subtree in chunk_tree.subtrees():

        if subtree.label() == "NP":
            phrase = " ".join(word for word, tag in
            subtree.leaves())
            noun_phrases.append(phrase)

        elif subtree.label() == "VP":
            phrase = " ".join(word for word, tag in
            subtree.leaves())
            verb_phrases.append(phrase)

    return noun_phrases, verb_phrases

# -----
# 4. Run on the dataset
# -----

all_noun_phrases = []
all_verb_phrases = []

for review in reviews:

    noun_phrases, verb_phrases =
    extract_phrases(review)

    all_noun_phrases.extend(noun_phrases)
    all_verb_phrases.extend(verb_phrases)

    print("\nSentence:")

```

```

print(review)

print("Noun phrases:")
print(noun_phrases)

print("Verb phrases:")
print(verb_phrases)

# -----
# 5. Simple keyphrase ranking
# -----

noun_phrase_frequency = Counter(
    phrase.lower()
    for phrase in all_noun_phrases
)

print("\nCandidate keyphrases:")

for phrase, frequency in
    noun_phrase_frequency.most_common():
    print(phrase, frequency)

```

星
0 / 5 ★

0

Topic C - table41

Task 1: B

“The lesson” is the direct object of “took,” and “to heart” is a separate prepositional phrase attached to the verb phrase.

Task 2: (d) and (e)

(d): “sacks of garbage” and “junk” are coordinated.

(e): “garbage” and “junk” are coordinated inside the phrase “sacks of garbage and junk.”

星
0 / 5 ★

0

Topic A Table 8 Task2

Background:

Named Entity Recognition (NER) aims to identify entities (e.g., persons, organizations, locations) and their types in text. Traditional approaches mainly rely on **sequence labeling** (BIO tagging) or **span extraction**, but they struggle with complex cases such as **nested entities** (entities inside other entities) and **discontinuous entities** (entities with separated words). Existing methods usually design different models for different NER settings, making unified NER challenging.

Key Idea:

The paper proposes **W2NER**, a new formulation that treats NER as a **word-word relation classification problem** instead of directly detecting entity spans.

Rather than asking:

"Which tokens form an entity?"

W2NER asks:

"What is the relationship between every pair of words?"

The model constructs a **2D word-pair grid**, where each cell represents the relationship between two words. It uses two main relations:

- **NNW (Next-Neighboring-Word)**: connects adjacent words inside an entity.
- **THW (Tail-Head-Word)**: identifies the connection between an entity's ending word and starting word, while also determining the entity type.

By predicting these relations, entities can be reconstructed as connected word graphs.

Model Architecture:

The framework consists of:

1. **BERT encoder** to obtain contextual word representations.
2. **BiLSTM** to enhance sequential information.
3. **2D grid representation** to model word-pair interactions.
4. **Multi-granularity 2D convolutions** to capture short- and long-range word relationships.
5. **Co-predictor (MLP + biaffine classifier)** to classify word-word relations.

Main Contributions:

1. **New NER formulation:**
Reframes NER from span detection into word-word relation classification.
2. **Unified NER framework:**
A single model can handle:
flat NER,
nested NER,
discontinuous NER.
3. **Strong performance:**
Achieves state-of-the-art results on 14 benchmark datasets, showing that relation-based modeling is effective for complex entity structures.

Core Insight:

The central contribution of W2NER is the shift from "**finding entity boundaries**" to "**reasoning about relationships between words**", allowing NER to be modeled as a structured graph prediction problem rather than a simple sequence labeling task.

星

5.0 / 5 ★ (1)

0

Task B - Table 21

POS tagging can be used for keyphrase extraction because important phrases often follow specific grammatical patterns. For example, many keyphrases are noun phrases such as "machine learning model" or "high image quality", while some useful expressions may be verb phrases such as "improve performance" or "reduce noise".

The basic idea is to first tokenize a sentence and assign a part-of-speech tag to each word. Then, we can define grammatical patterns using regular expressions. For example, a noun phrase may contain an adjective followed by one or more nouns, while a verb phrase may contain a verb followed by a noun phrase.

A simple example using NLTK is:

```
import nltk
text = "The new machine learning model improves image quality and reduces noise."
tokens = nltk.word_tokenize(text)
tagged = nltk.pos_tag(tokens)
grammar = r""" NP: {<JJ.*>*<NN.*>+} VP: {<VB.*>*<NP>} """
parser = nltk.RegexpParser(grammar)
tree = parser.parse(tagged)
for subtree in tree.subtrees():
    if subtree.label() in ["NP", "VP"]:
        phrase = " ".join(word for word, tag in subtree.leaves())
        print(subtree.label(), ":", phrase)
```

Here, NP represents noun phrases and VP represents verb phrases. The regular expression `<JJ.*>*<NN.*>+` means zero or more adjectives followed by one or more nouns. For example, it can extract "new machine learning model" and "image quality". Therefore, POS-based chunking provides a simple rule-based method for extracting keyphrases. Its advantage is that the rules are easy to understand and modify, although its performance depends on the quality of POS tagging and the grammatical patterns we define.

星

0 / 5 ★

0

Table 7 Topic A

Task 1:

1. Flat NER:

- Simplest and most classic NER
 - No entities contain or overlapped with another
 - Traditionally solved with sequence
 - **Cannot be used in a real world where sentence is more complex
- Eg. John is from USA. -> John is person and USA is location

2. Nested/Overlapping NER

- Entities shared token with other entities

- Nested is treated as special case of overlapped (fully contained sub-entity)
- common in domains with compound expression like biomedical text
- sequence labelling (one tag per token) breaks down here, since a single token might belong to multiple entities at once.
- Eg. Bank of China: China is a location and is nested inside organization (Bank of China)

3. Discontinuous NER

- An entity's words are not adjacent in the sentence
- hardest of the 3 types
- very common in clinical text
- traditional tagging schemes (BIO) can't naturally represent gaps, so extended schemes like BIOHD were invented

Task 2: Summary of W²NER (Li et al., AAAI 2022)20:32

Introduction: W²NER (AAAI 2022, Li et al.) unifies flat, nested, and discontinuous NER into a single framework by reframing NER as **word-word relation classification** instead of token tagging or span enumeration, solving the long-standing problem of needing separate models per NER type.

Core idea: For every pair of words in a sentence, the model predicts one of three relations — NONE, NNW (Next-Neighboring-Word, linking words that belong to the same entity, even non-adjacently), or THW-* (Tail-Head-Word, marking entity boundaries + type). Combining these two relation types lets one model reconstruct entities of any structural complexity.

Architecture:

- BERT + BiLSTM encoder produces contextualized word vectors
- A 2D word-pair grid is built via Conditional Layer Normalization, enriched with position/region embeddings
- Multi-granularity dilated CNNs refine the grid to capture near and distant word interactions
- A co-predictor (biaffine + MLP) classifies each word pair's relation
- A graph-search decoder (DFS) reconstructs entities from predicted NNW/THW edges

Strong points: Achieves new SOTA on 14 benchmarks (8 English, 6 Chinese) across all three NER types, while being ~3–5x faster and more parameter-efficient than span-based/Seq2Seq baselines — making it well-suited for real applications like clinical text mining (e.g., detecting discontinuous adverse-drug-reaction mentions).

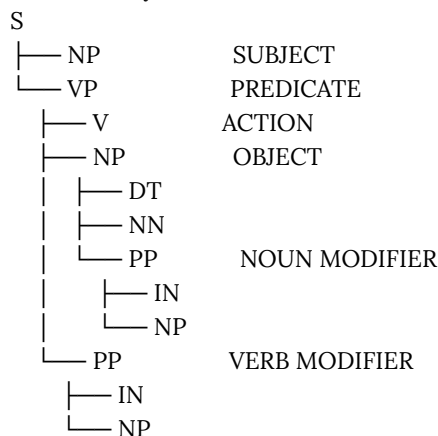
星

0 / 5 ★

0 0

⇒ Table 43 Topic C

The hierachy is



Task 1: Answer is B.

She= NP

took= VBD

the lesson= NP

to heart= PP

Therefore the priority should be S to NP(She) and VP, where VP further branches to VBD(took) NP(the lesson) and PP(to heart) which is most accurately described by option B.

Task 2: Answers (a) and (d) are correct because the phrase "sacks of garbage and junk" is structurally ambiguous. It can be interpreted as "[sacks of garbage] and junk]", where junk is a separate item being dumped, or as "sacks of [garbage and junk]", where both garbage and junk are contents of the sacks. Since both interpretations are grammatically valid, both parse trees are acceptable.

星

0 / 5 ★

0

TASK B - Table 23

In one line: tag each word's part of speech, then pull out patterns like "adjective + noun" — those are your keyphrases.

1. Tag each word. For "excellent battery life":
excellent = adjective, battery = noun, life = noun.
2. Keyphrases share the same shape. Useful phrases almost always look like "(adjective) + noun". A single word like "battery" is too vague, but "excellent battery life" carries the real meaning. Verb phrases start with a verb, e.g. "captures photos".
3. Use regex to catch that shape. Since the pattern is predictable, we write a rule. For example <DT>?<JJ>*<NN.*>+ means: (optional the/a) + (any adjectives) + (one or more nouns). Any words matching that order get grouped into a noun phrase. Why it works: no training or labeled data needed — just POS patterns to extract keyphrases.

output:

Review 1: The battery life of this laptop
is absolutely amazing and lasts all day.

Noun Phrases : ['The battery life', 'this laptop', 'all day']

Verb Phrases : ['is', 'lasts']

Review 2: I really love the crisp
display and the fast processor.

Noun Phrases : ['the crisp display', 'the fast processor']

Verb Phrases : ['love']

Review 3: The customer service completely
ignored my repeated complaints.

Noun Phrases : ['The customer
service', 'repeated complaints']

Verb Phrases : ['ignored']

Review 4: This wireless mouse has excellent
build quality but a short battery life.

Noun Phrases : ['This wireless mouse',
'excellent build quality', 'a short battery life']

Verb Phrases : ['has']

Review 5: The camera captures stunning
photos even in low light conditions.

Noun Phrases : ['The camera',
'photos', 'low light conditions']

Verb Phrases : ['captures stunning']

=====

=====

TOP KEY NOUN PHRASES:

1x The battery life

1x this laptop

```
import nltk
from nltk import pos_tag, word_tokenize, RegexpParser
from collections import Counter

for pkg in ["punkt", "punkt_tab", "averaged_perceptron_tagger",
            "averaged_perceptron_tagger_eng"]:
    try:
        nltk.download(pkg, quiet=True)
    except Exception:
        pass

reviews = [
    "The battery life of this laptop is absolutely amazing and lasts all day.",
    "I really love the crisp display and the fast processor.",
    "The customer service completely ignored my repeated complaints.",
    "This wireless mouse has excellent build quality but a short battery life.",
    "The camera captures stunning photos even in low light conditions.",
]

grammar = r"""
NP: {<DT>?<JJ>*<NN.*>+}
VP: {<VB.*>+<DT>?<JJ>*<NN.*>*}
"""
chunk_parser = RegexpParser(grammar)

def extract_phrases(text, phrase_label):
    tokens = word_tokenize(text)
    tagged = pos_tag(tokens)
    tree = chunk_parser.parse(tagged)
    phrases = []
    for subtree in tree.subtrees(filter=lambda t: t.label() == phrase_label):
        phrases.append(" ".join(word for word, tag in subtree.leaves()))

    return phrases
```

keyphrase_extraction

1x all day
1x the crisp display
1x the fast processor

星

0 / 5 ★

0 0

⇒ Topic C - table 34

For the first task correct answer is B. The phrase "to heart" is a prepositional phrase acting as an argument of the predicate "took" within the verb phrase, rather than a modifier of the noun phrase "the lesson". Tree B correctly attaches the PP "to heart" directly under the VP alongside the verb VBD ("took") and the direct object NP ("the lesson"). For the second task correct answers are a, b, d. Trees a and b represent the primary interpretation where the prepositional phrase "into a bin" modifies the main verb "dumped", and the conjunction "and" links "garbage" and "junk" within the object phrase. Tree a structures this coordination using standard NP nodes, while Tree b explicitly uses a conjoined noun phrase node. Tree d represents a valid alternative parse under higher-level coordination, where "and junk" attaches directly under the VP to form a conjoined object structure following the verb phrase "dumped sacks of garbage", while correctly attaching the directional PP "into a bin" to the main verb.

星

5.0 / 5 ★ (1)

0 0

⇒ Table 8-Topic A-task 1

Named Entity Recognition (NER) identifies entity boundaries and categories in text. Based on entity structure, NER can be divided into four types:

1. **Flat NER:** Entities are continuous and do not overlap. For example, "Barack Obama visited Paris" can be NER like [Barack Obama]PERSON visited [Paris]LOCATION. Each word belongs to at most one entity.
2. **Nested NER:** One entity is completely contained within another. For example, "Bank of China" can be NER like [Bank of [China]LOCATION]ORGANIZATION. "China" is a location, while "Bank of China" is an organization.
3. **Overlapped NER:** Two entities share some words without complete containment. In the abstract sequence "A B C," [A B]ENTITY-1 and [B C]ENTITY-2 share "B."
4. **Discontinuous NER:** One entity consists of separated text fragments. For example, in "aching in legs and shoulders," "aching in shoulders" combines the non-adjacent fragments "aching in" and "shoulders."

These four types describe entity structure, whereas labels such as PERSON and LOCATION describe semantic categories.

星

5.0 / 5 ★ (1)

0 0

⇒ Topic c-Table 33

1)b

"the lesson" forms a noun phrase, while "the heart" forms a preposition phrase. the preposition phrase "to heart" attaches to the verb phrase

2)a,b,d,e

The correct trees are (a), (b), (d), and (e) because the sentence has **structural ambiguity**, meaning that the words can be grouped into phrases in more than one valid way. For example, "garbage and junk" can form a coordinated noun phrase after "sacks of", giving the meaning that the sacks contain both garbage and junk, or "sacks of garbage" can be coordinated with "junk", meaning that the workers dumped two things: sacks of garbage and junk. The phrase "into a bin" can also be attached to the verb phrase "dumped sacks of garbage and junk", showing where the dumping action took place. Therefore, (a), (b), (d), and (e) represent valid constituent structures and interpretations, while (c) and (f) contain incorrect phrase attachments.

星

0 / 5 ★

0 1

To add on:

the phrase "**sacks of garbage and junk**" can be split into smaller, distinct Part of Speech (POS) units at three different levels:

Intermediate Sub-Phrases (Chunking)

You can group subsets of these words into smaller phrasal POS units (chunks) before building the full phrase:

"**garbage**" and "**junk**": Individual Noun Phrases

"**garbage and junk**": Coordinated Noun Phrase

"**of garbage and junk**": Prepositional Phrase

⇒ Table 11- Topic A

Task 1:NER can be broadly categorized into three major types according to the structural characteristics of entity mentions: **flat NER**, **overlapped NER**, and **discontinuous NER**. In **flat NER**, entity mentions are continuous spans of text and do not overlap with one another. **Overlapped NER** allows different entities to share one or more tokens and **nested NER** can be regarded as a special case of overlapped NER, in which one entity is completely contained within another. In contrast, **discontinuous NER** deals with entity mentions that consist of multiple non-adjacent text spans. These three types represent increasingly complex entity structures and are often studied under the framework of unified NER, which aims to recognize all of them using a single model.

星

0 / 5 ★

🗨 0

⇒ Table 11 (topic A)

Task 1: Different Types of NER and Their Properties

Flat NER refers to entities that are continuous and do not overlap with each other.This is the simplest type of named entity recognition.

Overlapped or Nested NER means that entities can overlap or one entity can be nested inside another.In this case, a single word may belong to more than one entity.

Discontinuous NER means that an entity is made up of non-adjacent word spans.

This is the most difficult type because the model needs to connect words that are not next to each other.

The W2NER paper proposes to solve all three types of NER in a unified way by modeling the relations between words.

It defines the NNW relation to connect neighboring words inside an entity.It also

defines the THW relation to identify the head and tail of an entity as well as its type.

By classifying these word-word relations, the model can handle flat, overlapped, and discontinuous entities at the same time.

Task 2:

The main contribution of this paper is a new perspective on NER. By focusing on modeling the relationships between words rather than only identifying boundaries, the authors successfully unify three different NER tasks into one elegant framework. The approach is both theoretically interesting and empirically strong.

星

0 / 5 ★

🗨 0

⇒ Topic C-table 31

1. B

2. D E

3. reference

code:<https://github.com/jyc114514/EE6405-class-discussion>

星

0 / 5 ★

🗨 0

jyc114514/EE6405-class-discussion



1Contributor

0Issues

0Stars

0Forks

GitHub

GitHub - jyc114514/EE6405-class-discussion

⇒ **TopicB :Table 15**

task1:POS tagging makes keyphrase extraction possible because keyphrases are not arbitrary word sequences — they follow predictable grammatical patterns, with topics expressed as noun phrases and actions as verb phrases. By tagging each token with its part of speech, we can flatten the sentence into a tag sequence such as <DT><JJ><JJ><NN> and then apply regular expressions over those tags rather than over the words themselves. A pattern like <DT>?<JJ.*>*<NN.*>+ chunks noun phrases such as "the cheap plastic case" and "excellent sound quality", while <MD>?<VB.*>*<RB.*>* chunks verb phrases such as "started cracking"; each match is then mapped back to its original token span to recover the phrase text. This shallow-parsing approach is fast, requires no training data, and yields linguistically well-formed candidates, though it only generates candidates — they are usually ranked afterwards by frequency or TF-IDF to select the final keyphrases, and determiners are typically stripped from the output. task2:in picture

星
0 / 5 ★

🗉 0

```
import re
from nltk import word_tokenize, pos_tag
import spacy

text = "The cheap plastic case started cracking after two weeks. Great headphones with excellent sound quality."

NP = re.compile(r'<DT>?<JJ.*>*<NN.*>+')
VP = re.compile(r'<MD>?<VB.*>*<RB.*>*')

def chunk(tagged, pattern):
    tag_str = ' '.join('%s' % tag for word, tag in tagged)
    phrases = []
    for m in re.finditer(pattern, tag_str):
        start = tag_str[m.start():].count('<')
        end = start + m.group().count('<')
        phrases.append(' '.join(w for w, t in tagged[start:end]))
    return phrases

nlp = spacy.load('en_core_web_sm')
tagged2 = [(token.text, token.tag_) for token in nlp(text)]
print('nltk POS:', chunk(tagged2, NP))
print('spacy VP:', chunk(tagged2, VP))
```

⇒ **Table 6 - Topic A - task 2**

W²NER formulates NER as a word-word relation classification problem. First, the input is encoded by BERT to obtain contextual representations, and then a BiLSTM is used to further capture bidirectional dependencies in the word sequence. Next, CLN combines the representations of each word pair to construct a 2D word-pair grid, while distance and region information are also incorporated and processed by convolution layers to model local interactions between word pairs. After that, the Co-Predictor combines sequence features and grid features to predict whether each word pair belongs to NNW, THW, or None. NNW connects neighboring words within the same entity, while THW links the tail word to the head word and also determines the entity type. Finally, during decoding, the model follows the predicted NNW paths to connect words and uses THW to determine whether a path forms a complete entity. The key idea is to predict structural relations between words first and then reconstruct entities from these relations, allowing the same framework to handle flat, nested, and discontinuous NER.

星
0 / 5 ★

🗉 0

⇒ **Table 37 (Topic C)**

Task 1:

The correct answer is option B. The phrase “to heart” belongs to the VP and forms part of the expression “take something to heart”, rather than being part of the noun phrase “the lesson”.

Task 2:

The correct answers are option D and E. This sentence has two possible interpretations because of the coordination with “and”:

D: [sacks of garbage] and [junk]

E: sacks of [garbage and junk]

星
5.0 / 5 ★ (3)

🗉 0

⇒ **task B - Table 21**

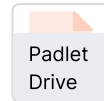
Sentence: I love this comfortable wireless keyboard because it works very well.
POS tags: [('I', 'PRP'), ('love', 'VBP'), ('this', 'DT'), ('comfortable', 'JJ'), ('wireless', 'NN'), ('keyboard', 'NN'), ('because', 'IN'), ('it', 'PRP'), ('works', 'VBZ'), ('very', 'RB'), ('well', 'RB'), (',', '.')]
Noun phrases: ['this comfortable wireless keyboard']
Verb phrases: ['love', 'works very well']

星
0 / 5 ★

🗨 0

↩ Topic B - Table 19

星
5.0 / 5 ★ (2)



🗨 1

Week2_TopicB_POS_Chunking

weijie010 Aug 21, 2026 08:47pm
Code and output are shown in this notebook.

↩ Topic C-table 39

Task1:Our answer is B.
My code is as follows:

```
import spacy
from spacy import displacy

nlp = spacy.load("en_core_web_sm")

sentence = 'She took the lesson to heart'

doc = nlp(sentence)

print("{:<15} | {:<10} | {:<15} | {:<20}".format('Token', 'Relation', 'Head', 'Children'))
print("-" * 70)
for token in doc:
    print("{:<15} | {:<10} | {:<15} | {:<20}".format(
        str(token.text),
        str(token.dep_),
        str(token.head.text),
        str([child for child in token.children])
    ))
```

Task2:Our answer is A and B
This follows the same code structure as Task 1, with only the sentence changed.
We think that In Task 2, the differing scopes of modification by 'sacks' led to two distinct answers.

星
5.0 / 5 ★ (2)

🗨 0

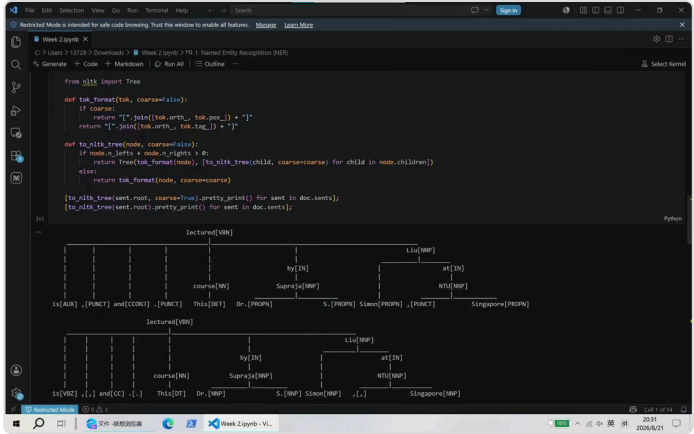
➡ Topic C Table 29

For Task 1, the correct answer is **B**. In the sentence “**She took the lesson to heart**,” the verb phrase is “**took the lesson to heart**.” The noun phrase “**the lesson**” is the object of “**took**,” and “**to heart**” is a prepositional phrase that also belongs to the verb phrase. This matches the expression “**take something to heart**.” Therefore, tree B has the correct structure.

For Task 2, the correct answers are **D and E**. In “**Workers dumped sacks of garbage and junk into a bin**,” the phrase “**into a bin**” modifies the verb “**dumped**,” so it should be attached to the verb phrase, not inside the noun phrase. There are also two possible interpretations of “**sacks of garbage and junk**.” It can mean “**sacks of [garbage and junk]**,” which corresponds to tree E, or “**[sacks of garbage] and [junk]**,” which corresponds to tree D. Therefore, both D and E are valid parses.

星
5.0 / 5 ★ (2)

🗨 0



➡ Table 13 _ Topic A

星
5.0 / 5 ★ (1)

🗨 0

Unified Named Entity Recognition: Types, Theory, and the W2NER Framework

Detailed study notes on "Unified Named Entity Recognition as Word-Word Relation Classification" (LL, Fel, Liu, Wu, Zhang, Zhao and Teng, AAAI 2022), with worked examples, formulas, and an architecture walkthrough.

1. Introduction: What is Named Entity Recognition?

Named Entity Recognition (NER) is the task of locating and classifying named entities in unstructured text. Given a sentence $s = w_1 w_2 \dots w_n$, an NER system must output the set of entities it contains. Each entity is a pair consisting of a contiguous or non-contiguous span of words and a semantic type (person, organization, location, symptom, gene, drug, etc.):

$$E(x) = \{ (s_1, \tau_1), (s_2, \tau_2), \dots, (s_n, \tau_n) \} \quad \text{where} \quad s = w_1 \dots w_n \quad (1 \leq i \leq n) \quad \text{and} \quad \tau_i \in T$$

Here T is a predefined type set and s is a span, i.e. a sequence of one or more words. NER is the foundation of information extraction: it feeds knowledge graphs, question answering, search engines, and — as discussed at the end of this document — clinical coding systems in healthcare.

The key observation of this document is that not all entities are simple sequences of adjacent words. Depending on how entity spans behave structurally, NER is traditionally divided into three families, and most classic models can only handle one of them. The W2NER framework (AAAI 2022) was proposed to handle all three in a single model.

2. Task 1: Three Types of NER and Their Properties

The three types differ in whether spans may touch, contain, or interleave each other. Figure 1 illustrates all three.

Padlet Drive ➡

NER与W2NER框架综述

➡ taskb 17

The main idea is actually quite simple.
First, POS tagging assigns a grammatical category to each word in a sentence. For example, a word can be tagged as a noun, adjective, verb, determiner, preposition, and so on.
For example, if we have the sentence:
“**The intelligent robot is performing complex tasks.**”
After POS tagging, “**intelligent**” is tagged as an adjective, “**robot**” as a noun, “**is**” as a verb, “**performing**” as a verb, “**complex**” as an adjective, and “**tasks**” as a plural noun.
The reason POS tagging is useful for keyphrase extraction is that many meaningful



POS_Tagging_Keyphrase_Extraction_Submission

phrases follow common grammatical patterns. For example, a noun phrase often contains an optional determiner, followed by one or more adjectives, and then one or more nouns. So in our example, we can extract phrases such as:

“the intelligent robot”

and

“complex tasks.”

These phrases are much more informative than extracting individual words such as “robot” or “tasks”.

To identify these patterns automatically, I

used **regular-expression-based chunking**.

In NLTK, the regular expressions are applied to POS tags instead of directly to characters in the text.

For noun phrases, I used a pattern that allows an optional determiner, followed by zero or more adjectives, and then one or more nouns.

For verb phrases, I used a pattern that allows an optional modal verb, followed by one or more verbs, and optionally some adverbs.

I also added a simple rule for prepositional phrases, where a preposition is followed by a noun phrase.

So the complete process can be summarized as:

Text, then tokenization, then POS

tagging, then regular-expression

chunking, and finally phrase extraction.

Now I will briefly explain the demo code.

First, I created a small sample dataset

containing several sentences related to robots, language models, and autonomous systems.

For each sentence, I first used `word_tokenize` to split the sentence into individual tokens.

Then I used `nltk.pos_tag` to

assign a POS tag to each token.

After that, I defined the chunking grammar using `RegexpParser`.

The parser looks at the sequence of POS tags and groups words that match the noun phrase, verb phrase, or prepositional phrase patterns.

For example, from the sentence:

“The intelligent robot is performing complex tasks in the laboratory.”

the system can identify noun phrases such as:

“The intelligent robot”

“complex tasks”

and

“the laboratory.”

It can also identify the verb phrase:

“is performing.”

These extracted phrases can then be treated as **candidate keyphrases**.

In this simple implementation, I mainly use noun phrases and verb phrases as candidate keyphrases because noun phrases often represent important objects, concepts, or topics, while verb phrases describe important actions or processes.

This approach has several advantages.

First, it is simple and easy to understand because all the extraction rules are explicitly defined.

Second, it does not require us to train a new model. We can directly use an existing POS tagger.

Third, it can preserve meaningful multi-word expressions instead of only extracting individual keywords. However, there are also some limitations. The performance depends on the accuracy of POS tagging, and manually designed grammar rules cannot cover every possible sentence structure. Also, a phrase that matches a grammatical pattern is only a **candidate keyphrase**. It does not necessarily mean that the phrase is actually important to the whole document. More advanced methods can therefore combine POS-based extraction with techniques such as **TF-IDF, frequency-based ranking, word embeddings, or neural models**.

星

0 / 5 ★

🗨 0

